TP2_ex3

November 4, 2026

1 Trabalho Prático — Multiplicação de Inteiros: SFOTS, k-indução e animação (Z3)

Este notebook contém: - a modelação do autómato (CFA) como SFOTS com `BitVec`'s de tamanho `n`, - um verificador por **k-indução** (caso base + passo) e um **BMC** auxiliar, - a verificação do invariante `x*y + z = a*b`, - a verificação de segurança (o estado de erro é inacessível com `a, b < 2^(n/2)`), - a propriedade de animação (terminação) provada com uma função variante (ranking), - discussão e limitações.

1.0.1 Decisões de modelação (justificação)

- **Estado:** `(pc, a, b, x, y, z)`, com `pc ∈ {init, skip, left, right, stop, error}` (tipo enumerado em Z3) e `a, b, x, y, z` do tipo `BitVec(n)`. `a` e `b` são constantes do programa: todas as transições mantêm-nas.
- **Estado inicial:** `pc = init ∧ x = a ∧ y = b ∧ z = 0`. A ação do nodo `init` (`x,y,z ← a,b,0`) fica codificada no próprio estado inicial. Assim o invariante `x*y + z = a*b` é válido em todos os estados, incluindo `init`.
- **Transições:** cada ramo do diagrama é uma aresta `(origem, destino, guarda, atualização)`. As guardas dos ramos que saem do mesmo nodo são mutuamente exclusivas e cobrem todos os casos. Por isso a precedência do erro sobre os outros ramos é automática: para `left` e `right`, o ramo de erro é o complemento exato do ramo normal.
- **Estados finais:** `stop` e `error` têm uma transição para si próprios (*stuttering*), para a relação de transição ser total e os caminhos poderem ter qualquer comprimento.
- **Overflow:** em Z3 as operações são totais e módulo `2^n`, logo o overflow tem de ser detetado explicitamente. Como os inputs são inteiros positivos, usa-se overflow **sem sinal**:
  - `x << 1` tem overflow sse o bit mais significativo de `x` é 1 (`x ≥ 2^(n-1)`);
  - `z + x` tem overflow (*carry*) sse `(z + x) < z` em ordem sem sinal.

  Com overflow com sinal, a hipótese `a, b < 2^(n/2)` (que garante `a*b < 2^n`) não chegaria para excluir o erro.
- **Operações sem sinal:** usa-se `LShR` para `y/2` e `ULT`/`UGE` para comparações.
- **Estado de erro:** quando há overflow, o estado seguinte guarda os valores "embrulhados" (módulo `2^n`) que o Z3 calcula.
- **k-indução:** o caso base procura contraexemplos em caminhos a partir do estado inicial. O passo procura `k+1` estados consecutivos em que a propriedade vale nos `k` primeiros e falha no último.
- **Invariantes auxiliares:** o invariante e a segurança não são 1-indutivos sozinhos, porque o passo parte de estados arbitrários (possivelmente inalcançáveis). Por isso usa-se:
  - `J`: `pc = left → (y ≠ 0 ∧ y par)` e `pc = right → y ímpar`;

– o invariante em aritmética de `2n` bits (sem *wrap-around*), para a segurança.

- **Animação (terminação):** prova-se com uma função variante `R(s)` que decresce estritamente em cada transição de um estado não final, mais a ausência de bloqueios (*deadlocks*). Complementa-se com uma verificação limitada (BMC) de que se chega a um estado final em `L` passos.

[1]:
```
## 1. Importações e parâmetros
import time
from z3 import (BitVec, BitVecVal, Const, EnumSort, Solver, BoolVal,
                And, Or, Not, Implies, If, ULT, UGE, LShR, ZeroExt, sat, unsat)

N = 8   # nº de bits (parâmetro do programa). Se ficar lento,fazer N = 6.
```

1.1 2. SFOTS do autómato (alínea 1)

Variáveis, estado inicial, relação de transição e estado de erro. `arestas(s)` devolve a lista de ramos do CFA; `trans(s, t)` é a disjunção das transições possíveis.

[2]:
```
PC, (INIT, SKIP, LEFT, RIGHT, STOP, ERROR) = EnumSort(
    "PC", ["init", "skip", "left", "right", "stop", "error"])


class SFOTS:
    def __init__(self, n):
        self.n = n

    # ---- variáveis do modelo (uma cópia por instante i) ----
    def estado(self, i):
        n = self.n
        return {"pc": Const(f"pc_{i}", PC),
                "a": BitVec(f"a_{i}", n), "b": BitVec(f"b_{i}", n),
                "x": BitVec(f"x_{i}", n), "y": BitVec(f"y_{i}", n), "z":␣
 ↪BitVec(f"z_{i}", n)}

    # ---- estado inicial (com a opção da hipótese a, b < 2^(n/2)) ----
    def init(self, s, limitar=False):
        c = [s["pc"] == INIT, s["x"] == s["a"], s["y"] == s["b"], s["z"] == 0]
        if limitar:
            lim = 2 ** (self.n // 2)
            c += [ULT(s["a"], lim), ULT(s["b"], lim)]
        return And(c)

    # ---- deteção de overflow (sem sinal) ----
    def ovf_left(self, s):
        return UGE(s["x"], 2 ** (self.n - 1))          # x << 1 perde o bit␣
 ↪mais significativo

    def ovf_right(self, s):
```

```
        return ULT(s["z"] + s["x"], s["z"])            # carry em z + x

    # ---- ramos do CFA: (nome, origem, destino, guarda, atualização) ----
    def arestas(self, s):
        x, y, z = s["x"], s["y"], s["z"]
        par = (y & 1) == 0
        ident = {"x": x, "y": y, "z": z}
        esq = {"x": x << 1, "y": LShR(y, 1), "z": z}
        dir_ = {"x": x, "y": y - 1, "z": z + x}
        return [
            ("init->skip",   INIT,  SKIP,  BoolVal(True),          ident),
            ("skip->left",   SKIP,  LEFT,  And(y != 0, par),       ident),
            ("skip->right",  SKIP,  RIGHT, And(y != 0, Not(par)),  ident),
            ("skip->stop",   SKIP,  STOP,  y == 0,                 ident),
            ("left->skip",   LEFT,  SKIP,  Not(self.ovf_left(s)),  esq),
            ("left->error",  LEFT,  ERROR, self.ovf_left(s),       esq),
            ("right->skip",  RIGHT, SKIP,  Not(self.ovf_right(s)), dir_),
            ("right->error", RIGHT, ERROR, self.ovf_right(s),      dir_),
            ("stop->stop",   STOP,  STOP,  BoolVal(True),          ident),
            ("error->error", ERROR, ERROR, BoolVal(True),          ident),
        ]

    # ---- relação de transição T(s, t) ----
    def trans(self, s, t):
        casos = []
        for _, src, dst, g, upd in self.arestas(s):
            casos.append(And(s["pc"] == src, g, t["pc"] == dst,
                             t["x"] == upd["x"], t["y"] == upd["y"], t["z"] ==␣
↪upd["z"],
                             t["a"] == s["a"], t["b"] == s["b"]))
        return Or(casos)

    # ---- estados de interesse ----
    def erro(self, s):
        return s["pc"] == ERROR

    def terminal(self, s):
        return Or(s["pc"] == STOP, s["pc"] == ERROR)

    # ---- propriedades ----
    def inv_modular(self, s):
        """x*y + z == a*b em aritmética módulo 2^n (alínea 2)."""
        return s["x"] * s["y"] + s["z"] == s["a"] * s["b"]

    def aux_J(self, s):
        """Invariante auxiliar: o ramo escolhido em skip é coerente com a␣
↪paridade de y."""
```

```
        return And(Implies(s["pc"] == LEFT, And(s["y"] != 0, (s["y"] & 1) ==␣
↪0)),
                   Implies(s["pc"] == RIGHT, (s["y"] & 1) == 1))

    def inv_forte(self, s):
        """Invariante indutivo para a segurança (alínea 3): sem erro, J,␣
↪limites de a e b
        e a igualdade x*y + z = a*b em 2n bits (sem wrap-around)."""
        n = self.n
        ext = lambda v: ZeroExt(n, v)
        lim = 2 ** (n // 2)
        return And(s["pc"] != ERROR,
                   self.aux_J(s),
                   ULT(s["a"], lim), ULT(s["b"], lim),
                   ext(s["x"]) * ext(s["y"]) + ext(s["z"]) == ext(s["a"]) *␣
↪ext(s["b"]))


M = SFOTS(N)
print("Arestas do CFA:")
for nome, *_ in M.arestas(M.estado(0)):
    print("  ", nome)
```

```
Arestas do CFA:
   init->skip
   skip->left
   skip->right
   skip->stop
   left->skip
   left->error
   right->skip
   right->error
   stop->stop
   error->error
```

1.2 3. Verificadores: k-indução e BMC

`k_inducao(M, P, k_max, limitar)`: para `k = 1, 2, ...` 1. **Caso base:** há um caminho de `k` estados a partir do estado inicial em que `P` falha? Se sim, `P` é falsa (contraexemplo). 2. **Passo:** há `k+1` estados consecutivos com `P` nos `k` primeiros e `¬P` no último? Se `unsat`, `P` é k-indutiva e, com o caso base, é um invariante.

`bmc(M, mau, profundidade, limitar)` procura um estado "mau" alcançável em até `profundidade` passos (serve para encontrar contraexemplos).

[3]:
```
def extrair_traco(m, S):
    traco = []
    for s in S:
```

```
        d = {"pc": str(m.eval(s["pc"], model_completion=True))}
        for v in "abxyz":
            d[v] = m.eval(s[v], model_completion=True).as_long()
        traco.append(d)
    return traco


def mostrar_traco(traco):
    for i, d in enumerate(traco):
        print(f"  s{i}: pc={d['pc']:<5} a={d['a']:<4} b={d['b']:<4} "
              f"x={d['x']:<4} y={d['y']:<4} z={d['z']}")


def k_inducao(M, P, k_max=8, limitar=False, verbose=True):
    falhas_passo = {}
    for k in range(1, k_max + 1):
        # --- caso base: estados s0..s_{k-1} a partir do estado inicial ---
        S = [M.estado(i) for i in range(k)]
        sol = Solver()
        sol.add(M.init(S[0], limitar))
        for i in range(k - 1):
            sol.add(M.trans(S[i], S[i + 1]))
        sol.add(Or([Not(P(s)) for s in S]))
        if sol.check() == sat:
            if verbose:
                print(f"k={k}: caso base FALHA -> a propriedade é falsa")
            return {"resultado": "FALSIFICADA", "k": k,
                    "contraexemplo": extrair_traco(sol.model(), S),␣
 ↪"falhas_passo": falhas_passo}

        # --- passo de indução: s0..sk ---
        T = [M.estado(i) for i in range(k + 1)]
        sol = Solver()
        for i in range(k):
            sol.add(M.trans(T[i], T[i + 1]))
            sol.add(P(T[i]))
        sol.add(Not(P(T[k])))
        r = sol.check()
        if r == unsat:
            if verbose:
                print(f"k={k}: base OK, passo OK -> propriedade PROVADA por␣
 ↪{k}-indução")
            return {"resultado": "PROVADA", "k": k, "falhas_passo":␣
 ↪falhas_passo}
        falhas_passo[k] = extrair_traco(sol.model(), T) if r == sat else None
        if verbose:
```

```
            print(f"k={k}: base OK, passo FALHA (contraexemplo ao passo; estado␣
 ↪inicial pode ser inalcançável)")
    return {"resultado": "INCONCLUSIVO", "k": k_max, "falhas_passo":␣
 ↪falhas_passo}


def bmc(M, mau, profundidade, limitar=False):
    """Procura um estado 'mau' alcançável em L <= profundidade passos."""
    for L in range(profundidade + 1):
        S = [M.estado(i) for i in range(L + 1)]
        sol = Solver()
        sol.add(M.init(S[0], limitar))
        for i in range(L):
            sol.add(M.trans(S[i], S[i + 1]))
        sol.add(mau(S[L]))
        if sol.check() == sat:
            return L, extrair_traco(sol.model(), S)
    return None


def executar(M, a, b, passos):
    """Execução concreta do SFOTS (é determinista) para sanity check."""
    S = [M.estado(i) for i in range(passos + 1)]
    sol = Solver()
    sol.add(M.init(S[0]), S[0]["a"] == a, S[0]["b"] == b)
    for i in range(passos):
        sol.add(M.trans(S[i], S[i + 1]))
    assert sol.check() == sat
    tr = extrair_traco(sol.model(), S)
    for i, d in enumerate(tr):
        print(f"  s{i}: pc={d['pc']:<5} x={d['x']:<4} y={d['y']:<4} z={d['z']}")
        if d["pc"] in ("stop", "error"):
            break
    return tr
```

[4]:
```
## Sanity check do modelo: executar 7 * 5 com n = 8
tr = executar(M, 7, 5, 4 * N + 2)
```

```
  s0: pc=init  x=7    y=5    z=0
  s1: pc=skip  x=7    y=5    z=0
  s2: pc=right x=7    y=5    z=0
  s3: pc=skip  x=7    y=4    z=7
  s4: pc=left  x=7    y=4    z=7
  s5: pc=skip  x=14   y=2    z=7
  s6: pc=left  x=14   y=2    z=7
  s7: pc=skip  x=28   y=1    z=7
  s8: pc=right x=28   y=1    z=7
  s9: pc=skip  x=28   y=0    z=35
```

```
  s10: pc=stop  x=28   y=0    z=35
```

1.3 4. Alínea 2 — `x*y + z = a*b` é um invariante?

Propriedade em aritmética módulo 2^n: `x*y + z == a*b`.

Análise manual: o passo `left` só preserva a propriedade se `y` for par (com `y` ímpar, `y>>1 = (y-1)/2` e a igualdade quebra). Com `k = 1`, o passo pode começar num estado inalcançável com `pc = left` e `y` ímpar. Com `k = 2`, esse estado teria de ser precedido por `skip → left`, cuja guarda exige `y` par, o que é impossível. Espera-se, por isso, que `k = 1` falhe no passo e `k = 2` prove a propriedade. **Confirmar na execução.**

[5]:
```
t0 = time.perf_counter()
r2 = k_inducao(M, M.inv_modular, k_max=6)
print("Resultado:", r2["resultado"], "| k =", r2["k"], f"| {time.perf_counter()␣
 ↪- t0:.2f}s")

if 1 in r2["falhas_passo"] and r2["falhas_passo"][1]:
    print("\nContraexemplo ao passo com k = 1:")
    mostrar_traco(r2["falhas_passo"][1])
```

```
k=1: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
k=2: base OK, passo OK -> propriedade PROVADA por 2-indução
Resultado: PROVADA | k = 2 | 358.08s

Contraexemplo ao passo com k = 1:
  s0: pc=left  a=81   b=185  x=139  y=1    z=254
  s1: pc=error a=81   b=185  x=22   y=0    z=254
```

1.4 5. Alínea 3 — Segurança: o estado de erro é inacessível

1.4.1 5.1 Sem a hipótese há erros (BMC)

Sem restringir `a` e `b`, o overflow é possível: o BMC deve encontrar um caminho para `error`.

[6]:
```
res = bmc(M, M.erro, 4 * N + 2, limitar=False)
if res:
    L, tr = res
    print(f"Erro alcançável em {L} passos sem a hipótese. Contraexemplo:")
    mostrar_traco(tr)
else:
    print("Nenhum erro encontrado até à profundidade pedida.")
```

```
Erro alcançável em 3 passos sem a hipótese. Contraexemplo:
  s0: pc=init  a=136  b=224  x=136  y=224  z=0
  s1: pc=skip  a=136  b=224  x=136  y=224  z=0
  s2: pc=left  a=136  b=224  x=136  y=224  z=0
  s3: pc=error a=136  b=224  x=16   y=112  z=0
```

1.4.2 5.2 Com a, b < 2^(n/2): BMC e k-indução

- **BMC** (apenas evidência, não prova): não deve haver erro até à profundidade 4n+2.
- **k-indução da propriedade simples** pc error: espera-se que o passo falhe, porque parte de estados arbitrários (por exemplo pc = left com x 2^(n-1)), que não satisfazem as relações entre x, y, z, a, b. Por isso é preciso um invariante mais forte.
- **k-indução do invariante forte** inv_forte: inclui pc error, J, os limites de a e b e x*y + z = a*b em 2n bits.
  - Se o overflow ocorresse em left, teríamos x 2^(n-1) e y 2, logo x*y 2^n > a*b, o que contradiz a igualdade.
  - Se ocorresse em right, teríamos x + z 2^n, mas x x*y (y 1), logo x + z a*b < 2^n, o que também contradiz.
  - O estado inicial (com a hipótese) satisfaz o invariante.

Como inv_forte pc error, a prova do invariante forte garante a segurança.

[7]:
```
# BMC com a hipótese (evidência)
res = bmc(M, M.erro, 4 * N + 2, limitar=True)
print("BMC com hipótese:", "erro encontrado!" if res else "sem erro até␣
 ↪profundidade", 4 * N + 2)

# k-indução da propriedade simples (esperado: não é k-indutiva para k pequeno)
nao_erro = lambda s: s["pc"] != ERROR
print("\n--- k-indução de pc != error ---")
r_simples = k_inducao(M, nao_erro, k_max=6, limitar=True)
print("Resultado:", r_simples["resultado"])
```

```
BMC com hipótese: sem erro até profundidade 34

--- k-indução de pc != error ---
k=1: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
k=2: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
k=3: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
k=4: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
k=5: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
k=6: base OK, passo FALHA (contraexemplo ao passo; estado inicial pode ser
inalcançável)
Resultado: INCONCLUSIVO
```

[8]:
```
# k-indução do invariante forte (com a hipótese no estado inicial)
print("--- k-indução do invariante forte ---")
t0 = time.perf_counter()
r3 = k_inducao(M, M.inv_forte, k_max=4, limitar=True)
```

```
print("Resultado:", r3["resultado"], "| k =", r3["k"], f"| {time.perf_counter()␣
 ↪- t0:.2f}s")

if r3["resultado"] == "PROVADA":
    print(f"\nSEGURANÇA: com a < 2^{N // 2} e b < 2^{N // 2}, o estado de erro␣
 ↪é inacessível (n = {N}).")
```

```
--- k-indução do invariante forte ---
k=1: base OK, passo OK -> propriedade PROVADA por 1-indução
Resultado: PROVADA | k = 1 | 5.08s

SEGURANÇA: com a < 2^4 e b < 2^4, o estado de erro é inacessível (n = 8).
```

1.4.3 5.3 (Extra) Correção funcional

Com o mesmo invariante forte mais `pc = stop → (y = 0  z = a*b)`, prova-se que, quando o programa pára, `z = a*b`.

[9]:
```
corretude = lambda s: And(M.inv_forte(s),
                          Implies(s["pc"] == STOP, And(s["y"] == 0, s["z"] ==␣
 ↪s["a"] * s["b"])))
r_corr = k_inducao(M, corretude, k_max=4, limitar=True)
print("Resultado:", r_corr["resultado"], "| k =", r_corr["k"])
```

```
k=1: base OK, passo OK -> propriedade PROVADA por 1-indução
Resultado: PROVADA | k = 1
```

1.5 6. Alínea 4 — Propriedade de animação (terminação)

Propriedade: para quaisquer valores iniciais de `a` e `b`, toda a execução chega a um estado final (`stop` ou `error`). Com a hipótese `a, b < 2^(n/2)` o estado final é `stop`.

Prova (três passos):

1. **Invariante auxiliar** `J` (1-indutivo): `left  y  0  y par` e `right  y ímpar`.
2. **Ausência de bloqueios:** em todo o estado não final que satisfaz `J`, pelo menos uma guarda está ativa.
3. **Função variante** `R(s) = 2*y + c(pc)`, com `c(init)=2`, `c(skip)=1` e `c(left)=c(right)=c(stop)=c(error)=0`. Tem valores em (calculada em `n+2` bits sem sinal, logo sem overflow) e decresce estritamente em cada transição de um estado não final, assumindo `J`:
   - `init→skip`: `2y+2 → 2y+1`; `skip→left/right`: `2y+1 → 2y`; `skip→stop`: `1 → 0`;
   - `left→skip`: `2y → y+1` (decresce porque `y  2`); `right→skip`: `2y → 2y−1`;
   - as transições para `error` também decrescem.

Como é bem fundado, não há execuções infinitas, e sem bloqueios todas terminam num estado final. Como verificação complementar, o BMC mostra que um estado final é atingido em `L = 4n+2` passos (o pior caso estimado é `4n`).

[10]:
```
def rank(M, s):
    w = M.n + 2
    c = If(s["pc"] == INIT, BitVecVal(2, w),
           If(s["pc"] == SKIP, BitVecVal(1, w), BitVecVal(0, w)))
    return ZeroExt(2, s["y"]) * 2 + c


# 1) J é invariante (1-indutivo), para quaisquer a, b
rJ = k_inducao(M, M.aux_J, k_max=3, limitar=False, verbose=False)
print("1) J invariante:", rJ["resultado"], "k =", rJ["k"])

# 2) Ausência de bloqueios
s = M.estado("s")
cobertura = Or([And(s["pc"] == src, g) for (_, src, _, g, _) in M.arestas(s)])
sol = Solver()
sol.add(M.aux_J(s), Not(M.terminal(s)), Not(cobertura))
print("2) Sem bloqueios (unsat esperado):", sol.check())

# 3) R decresce estritamente em cada transição de um estado não final
s, t = M.estado("s"), M.estado("t")
sol = Solver()
sol.add(M.aux_J(s), Not(M.terminal(s)), M.trans(s, t),
        Not(ULT(rank(M, t), rank(M, s))))
r_rank = sol.check()
print("3) R decresce sempre (unsat esperado):", r_rank)

if rJ["resultado"] == "PROVADA" and r_rank == unsat:
    print("\nANIMAÇÃO: o programa termina para quaisquer a, b (estado final: stop ou error).")
```

```
1) J invariante: PROVADA k = 1
2) Sem bloqueios (unsat esperado): unsat
3) R decresce sempre (unsat esperado): unsat

ANIMAÇÃO: o programa termina para quaisquer a, b (estado final: stop ou error).
```

[11]:
```
# Verificação complementar (BMC): estado final em L passos, para todos os a, b
def termina_em(M, L, limitar=False, alvo=None):
    S = [M.estado(i) for i in range(L + 1)]
    sol = Solver()
    sol.add(M.init(S[0], limitar))
    for i in range(L):
        sol.add(M.trans(S[i], S[i + 1]))
    final = M.terminal(S[L]) if alvo is None else S[L]["pc"] == alvo
    sol.add(Not(final))
    return sol.check()

L = 4 * N + 2
```

```
print(f"Qualquer a, b termina (stop ou error) em <= {L} passos? (unsat = sim):",
      termina_em(M, L))
print(f"Com a, b < 2^{N // 2}, chega-se a 'stop' em <= {L} passos? (unsat =␣
 ↪sim):",
      termina_em(M, L, limitar=True, alvo=STOP))
```

```
Qualquer a, b termina (stop ou error) em <= 34 passos? (unsat = sim): unsat
Com a, b < 2^4, chega-se a 'stop' em <= 34 passos? (unsat = sim): unsat
```

1.6 7. Conclusões e discussão

- **Modelo:** o CFA foi codificado como SFOTS com `BitVec(n)`. O overflow tem de ser programado explicitamente, porque o Z3 é total e módulo `2^n`. Optou-se por overflow sem sinal (carry), coerente com inteiros positivos e com a hipótese `a, b < 2^(n/2)`.
- **Invariante (alínea 2):** `x*y + z = a*b` (módulo `2^n`) é k-indutivo com `k = 2` (resultado a confirmar na execução). Com `k = 1` o passo falha por causa de estados inalcançáveis com `pc = left` e `y` ímpar.
- **Segurança (alínea 3):** a propriedade `pc  error` não é k-indutiva por si só, por isso foi necessário um invariante mais forte (aritmética de `2n` bits mais `J`). Sem a hipótese há erros alcançáveis (contraexemplo do BMC). Com ela, o erro é inacessível.
- **Animação (alínea 4):** a prova usa o invariante `J`, a ausência de bloqueios e uma função variante `R`. O BMC confirma o limite de passos.
- **Limitações:**
  - a prova é feita para um `n` concreto (`N`);
  - a multiplicação em `2n` bits pode tornar o Z3 lento para `n` grande;
  - os resultados dependem da máquina e da versão do Z3.

1.6.1 Declaração de utilização de IA

O código-base foi gerado com apoio de IA a partir do enunciado, e usei outra IA para reconstruir o código por fases. Eu: - executei o notebook completo com N = 8 e obtive os resultados indicados acima; - verifiquei os contraexemplos à mão (por exemplo, o da alínea 2 com k = 1); - detetei e corrigi um erro de indentação do `return tr` na função `executar`;